Extracted Codes (12 Programs)

1. Insertion Sort 1

```
#include <stdio.h>
void print(int ar_size, int* ar) {
    int i;
    for(i=0; i<ar_size; i++) {
        printf("%d " , ar[i]);
    }
    printf("\n");
}

#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <assert.h>

/* Head ends here */
void insertionSort(int ar_size, int *  ar) {
    int j = ar_size-1;
    int v = ar[j];
    while(v < ar[j-1]) {
        ar[j] = ar[j-1];
        j--;
        print(ar_size, ar);
    }
    ar[j] = v;
    print(ar_size, ar);
}

/* Tail starts here */
int main() {
   int _ar_size;
   scanf("%d" , &_ar_size);
   int _ar[_ar_size], _ar_i;
   for(_ar_i = 0; _ar_i < _ar_size; _ar_i++) {
      scanf("%d" , &_ar[_ar_i]);
   }

   insertionSort(_ar_size, _ar);
```

```
   return 0;
}
```

2. Insertion Sort 2

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <assert.h>

void insertionSort(int ar_size, int *  ar) {
    for (int i = 1; i < ar_size; ++i) {
        int j = i - 1;
        int p = ar[i];
        while (j >= 0 && p < ar[j]) {
            ar[j+1] = ar[j];
            j--;
        }
        ar[j+1] = p;
        printf("%d" , ar[0]);
        for (int k = 1; k < ar_size; ++k) {
            printf(" %d" , ar[k]);
        }
        printf("\n");
    }
}

int main() {
   int _ar_size;
   scanf("%d" , &_ar_size);
   int _ar[_ar_size], _ar_i;
   for(_ar_i = 0; _ar_i < _ar_size; _ar_i++) {
      scanf("%d" , &_ar[_ar_i]);
   }
   insertionSort(_ar_size, _ar);
   return 0;
}
```

3. Compare the Triplets

```
#include <math.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <assert.h>
#include <limits.h>
#include <stdbool.h>

int main(){
    int a0;
    int a1;
    int a2;

    scanf("%d %d %d",&a0,&a1,&a2);

    int b0;
    int b1;
    int b2;

    scanf("%d %d %d",&b0,&b1,&b2);

    int a_score = 0, b_score = 0;

    if(a0 > b0) a_score++;
    else if(b0 > a0) b_score++;

    if(a1 > b1) a_score++;
    else if(b1 > a1) b_score++;

    if(a2 > b2) a_score++;
    else if(b2 > a2) b_score++;

    printf("%d %d",a_score, b_score);
    return 0;
```

}

4. Diagonal Difference

```
#include <math.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```
#include <assert.h>
#include <limits.h>
#include <stdbool.h>

int main()
{
    int n, j;
    int i=0, RightDiagonalSum=0, LeftDiagonalSum=0,
firstarray, secondarray;
    scanf("%d",&n);
    int a[n][n];
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }
    for(i = 0; i < n; i++) {
        RightDiagonalSum += a[i][i];
        LeftDiagonalSum += a[i][n - 1 - i];
    }
    printf("%d",abs(RightDiagonalSum-LeftDiagonalSum));
    return 0;
}
```

5. Counting Sort 1

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>

int main() {
    int n,i;
    int b[100],a;
    scanf("%d" ,&n);
    for(i=0;i<100;i++) b[i]=0;
    for(i=0;i<n;i++) { scanf("%d" ,&a); b[a]++; }
    for(i=0;i<100;i++) printf("%d ", b[i]);
    return 0;
}
```

6. Non Divisible Subset

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// Helper functions for min and max
int min(int a, int b) {
    return a < b ? a : b;
}
int max(int a, int b) {
    return a > b ? a : b;
}

int main() {
    int n, k, a, total = 0;

    // Read input n and k
    scanf("%d %d", &n, &k);

    int arr[n];
    int count[k];

    for(int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    for(int i = 0; i < k; i++)
        count[i] = 0;

    for(int i = 0; i < n; i++)
        count[arr[i] % k]++;

    if(count[0] > 0)
        total++;

    for(int i = 1; i <= k/2; i++) {
        if(i == k - i) {
            if(count[i] > 0)
                total++;
        } else {
            total += max(count[i], count[k - i]);
        }
    }
```

```
    // Print the size of the largest non-divisible subset
    printf("%d\n", total);

    return 0;
}
```

7. Counter Game

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
int isPow2(long unsigned int);
unsigned long int largePow(long unsigned int);

int main() {
    int t,i,win;
    long unsigned int n;
    scanf("%d" ,&t);
    for(i=0;i<t;++i) {
        win=0;
        scanf("%lu" ,&n);
        if(n==1) printf("Richard\n");
        else {
            while(n!=1) {
                if(isPow2(n)) n>>=1;
                else n-=largePow(n);
                ++win;
            }
        }
        if(win%2==0) printf("Richard\n");
        else printf("Louise\n");
    }
    return 0;
}
int isPow2(long unsigned int n) { return !(n&(n-1)); }
long unsigned int largePow(long unsigned int n) {
    long unsigned int m;
    while(n) { m=n; n=n&(n-1); }
    return m;
}
```

8. Bear and Steady Gene

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>

// Function to check if all characters outside the window are balanced
(<= n/4)
int is_balanced(int count[], int target)
{
    if (count['A'] > target || count['C'] > target || count['G'] >
target || count['T'] > target) {
        return 0; // Not balanced
    }
    return 1; // Balanced
}

int steadyGene(char* gene, int n) {

    int count[256] = {0};
    // Count frequency of each character
    for(int i = 0; i < n; i++) {
        count[gene[i]]++;
    }
    int target = n / 4;
    // If already steady
    if(is_balanced(count, target))
        return 0;
    int min_len = n;
    int left = 0;
    // Sliding window: right pointer expands the window
    for(int right = 0; right < n; right++) {
        count[gene[right]]--;
        while(is_balanced(count, target)) {
            if(right - left + 1 < min_len)
                min_len = right - left + 1;
            count[gene[left]]++;
            left++;
        }
    }

    return min_len;
}
```

```
int main() {
    int n;
    // Read the gene length
    if (scanf("%d", &n) != 1) return 1;
    char* gene = (char*)malloc((n + 1) * sizeof(char));
    // Read the gene string
    if (scanf("%s", gene) != 1) {
        free(gene);
        return 1;
    }
    int result = steadyGene(gene, n);
    printf("%d\n", result);
    free(gene);
    return 0;
}
```

9. Ceaser Cipher (Caesar Cipher)

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>

int main() {
    int n,i,j,k;
    char ar[101];
    unsigned char x;
    scanf("%d" ,&n);
    scanf("%s" ,ar);
    scanf("%d" ,&k);
    for(i=0;i<n;i++) {
        x=ar[i];
        if(x>=97 && x<=122) {
            x=x+(k%26);
            if(x>122) x=96+(x-122);
            ar[i]=x;
        } else if(x>=65 && x<=90) {
            x=x+(k%26);
            if(x>90) x=64+(x-90);
            ar[i]=x;
        }
    }
    printf("%s" ,ar);
```

```
    return 0;
}
```

10. Pangrams

```
#include <stdio.h>
char s[10000];
int main() {
    gets(s);
    int f[300]={0},ans=0,i;
    int l=strlen(s);
    for(i=0;i<l;i++) {
        if(s[i]==' ') continue;
        if(s[i]>='a') s[i]=s[i]-('a'-'A');
        if(!(f[s[i]]++)) ans++;
    }
    if(ans!=26) printf("not ");
    printf("pangram\n");
    return 0;
}
```

11. Closest Number

```
#include<stdio.h>
#include<stdlib.h>

void quickSort(int[], int, int);
int partition(int[], int, int);

int main(){
 int i,n;
 scanf("%d" ,&n);
 int *a=(int *)malloc(n*sizeof(int));
 for(i=0;i<n;i++) scanf("%d" ,&a[i]);
 quickSort(a,0,n-1);
 int min=a[1]-a[0];
 for(i=2;i<n;i++) if(a[i]-a[i-1]<min) min=a[i]-a[i-1];
 for(i=1;i<n;i++) if(a[i]-a[i-1]==min) printf("%d %d " ,a[i-1],a[i]);
 printf("\n");
 return 0;
}

void quickSort(int a[], int l, int r) {
   int j;
   if( l < r ) {
```

```
      j = partition( a, l, r);
      quickSort( a, l, j-1);
      quickSort( a, j+1, r);
   }
}

int partition(int a[], int l, int r) {
   int pivot, i, j, t;
   pivot = a[l];
   i = l; j = r+1;
   while(1) {
      do ++i; while( a[i] <= pivot && i <= r );
      do --j; while( a[j] > pivot );
      if( i >= j ) break;
      t = a[i]; a[i] = a[j]; a[j] = t;
   }
   t = a[l]; a[l] = a[j]; a[j] = t;
   return j;
}
```

12. Icecream Parlor

```
#include<stdio.h>
int main() {
    int t,c,l,i,j,arr[20000];
    scanf("%d" ,&t);
    for( ; t>0 ; t--) {
        scanf("%d%d" ,&c,&l);
        for(i=0;i<l;i++) scanf("%d" ,&arr[i]);
        for(i=0;i<l-1;i++)
            for(j=i+1;j<l;j++) {
                if(arr[i]+arr[j]==c) printf("%d %d\n" ,i+1,j+1);
            }
    }
    return 0;
}
```